



Value Types

Concepts

Value Types

NautilusTrader provides specialized value types for representing core trading concepts: `Price`, `Quantity`, and `Money`. These types use fixed-point arithmetic internally for performant, deterministic calculations across different platforms and environments.

Overview

Type	Purpose	Signed	Currency
<code>Quantity</code>	Trade sizes, order amounts, positions.	No	-
<code>Price</code>	Market prices, quotes, price levels.	Yes	-
<code>Money</code>	Monetary amounts, P&L, account balances.	Yes	Yes

Immutability

All value types are immutable. Once a value is constructed, it cannot be changed. Operations do not mutate the original object.

```
from nautilus_trader.model.objects import Quantity

qty1 = Quantity(100, precision=0)
qty2 = Quantity(50, precision=0)

# This creates a NEW Quantity; qty1 and qty2 are unchanged
result = qty1 + qty2

print(qty1)    # 100
print(qty2)    # 50
```

```
print(result)  # 150
```

This design provides several benefits:

- **Thread safety:** Immutable values can be safely shared across threads without synchronization.
- **Predictability:** Values never change unexpectedly, making debugging easier.
- **Hashability:** Immutable types can be used as dictionary keys and in sets.

Arithmetic operations

Value types support standard arithmetic operators (+, -, *, /, %, //) and unary operators (-, +, abs). The return type depends on the operator and the operand types.

Same-type binary operations

Addition and subtraction of the same value type return that type, preserving domain meaning (a price plus a price is still a price):

Operation	Result
Quantity + Quantity	Quantity
Quantity - Quantity	Quantity
Price + Price	Price
Price - Price	Price
Money + Money	Money
Money - Money	Money

```
from nautilus_trader.model.objects import Price

price1 = Price(100.50, precision=2)
price2 = Price(0.25, precision=2)
```



```
result = price1 + price2 # Returns Price(100.75, precision=2)
print(type(result))      # <class 'Price'>
```

Multiplication, division, floor division, and modulo between two values of the same type return `Decimal`:

Operation	Result
<code>Price * Price</code>	<code>Decimal</code>
<code>Price / Price</code>	<code>Decimal</code>
<code>Price // Price</code>	<code>Decimal</code>
<code>Price % Price</code>	<code>Decimal</code>

The same pattern applies to `Quantity` and `Money`.

These operations do not return the original type because the result has different dimensional meaning. Multiplying a price by a price produces "price squared", not a price. Dividing a quantity by a quantity produces a dimensionless ratio, not a quantity. Returning `Decimal` makes the unit change explicit and prevents misinterpretation of the result as a value with the original unit.

Unary operations

Unary operators preserve the value type where the result is valid for that type:

Operation	<code>Price</code>	<code>Quantity</code>	<code>Money</code>
<code>-x</code> (neg)	<code>Price</code>	<code>Decimal</code>	<code>Money</code>
<code>+x</code> (pos)	<code>Price</code>	<code>Quantity</code>	<code>Money</code>
<code>abs(x)</code>	<code>Price</code>	<code>Quantity</code>	<code>Money</code>
<code>int(x)</code>	<code>int</code>	<code>int</code>	<code>int</code>
<code>float(x)</code>	<code>float</code>	<code>float</code>	<code>float</code>

Operation	Price	Quantity	Money
<code>round(x)</code>	Decimal	Decimal	Decimal

`Quantity.__neg__` returns `Decimal` rather than `Quantity` because `Quantity` is unsigned and cannot represent a negative value.

```
from nautilus_trader.model.objects import Price, Quantity, Money
from nautilus_trader.model.currencies import USD

price = Price(100.50, precision=2)
print(-price)          # -100.50
print(type(-price))    # <class 'Price'>

money = Money(-50.00, USD)
print(abs(money))       # 50.00 USD
print(type(abs(money))) # <class 'Money'>

qty = Quantity(10, precision=0)
print(+qty)             # 10
print(type(+qty))       # <class 'Quantity'>
```

Mixed-type operations

When operating with other numeric types, the result type follows Python's [numeric tower](#) conventions. The general principle is that operations widen to the more general type: `float` operations return `float`, while `int` and `Decimal` operations return `Decimal` for precision preservation.

This applies to all six binary operators (`+`, `-`, `*`, `/`, `//`, `%`) and works in both directions (`value op scalar` and `scalar op value`):

Left operand	Right operand	Result type
Value type	<code>int</code>	<code>Decimal</code>
Value type	<code>float</code>	<code>float</code>
Value type	<code>Decimal</code>	<code>Decimal</code>

Left operand	Right operand	Result type
<code>int</code>	Value type	<code>Decimal</code>
<code>float</code>	Value type	<code>float</code>
<code>Decimal</code>	Value type	<code>Decimal</code>

```

from decimal import Decimal
from nautilus_trader.model.objects import Quantity

qty = Quantity(100, precision=0)

# Quantity + int -> Decimal
result1 = qty + 50
print(type(result1)) # <class 'decimal.Decimal'>

# Quantity + float -> float
result2 = qty + 50.5
print(type(result2)) # <class 'float'>

# Quantity + Decimal -> Decimal
result3 = qty + Decimal("50")
print(type(result3)) # <class 'decimal.Decimal'>

```

Precision handling

Each value type stores a precision field indicating the number of decimal places. Precision is set at construction and is immutable. There is no "unspecified" precision.

Fixed-point representation

Value types are stored internally as integers scaled to a global fixed precision (e.g., 10^{16} in high-precision mode), not floating-point numbers. The `precision` field tracks the number of decimal places used at construction, controlling display formatting and serialization, but the underlying raw value always uses the global scale.

```

from nautilus_trader.model.objects import Price

p1 = Price(1.23, precision=2) # displays as "1.23"

```

```
p2 = Price(1.230, precision=3) # displays as "1.230"

p1 == p2 # True: same underlying value
str(p1)  # "1.23"
str(p2)  # "1.230"
```

Precision controls display, not identity. Two prices with the same decimal value but different precisions are equal. The `precision` field determines string formatting and how many decimal places are shown, but equality is based on the underlying numeric value.

Market data serialization uses precision metadata. When market data types (quotes, trades, order book deltas) are written to Parquet or Arrow format, precision is stored in the file metadata so that values can be correctly decoded. All market data values within a single file must share the same precision.

 If a venue changes an instrument's tick size (and thus its precision), data files written before and after the change will have different precision metadata and should not be consolidated into a single file.

For how instrument-level precision constrains valid prices and quantities, see the [Precision](#) section of the Instruments guide.

Arithmetic precision

When performing arithmetic between values with different precisions, the result uses the maximum precision of the operands.

```
from nautilus_trader.model.objects import Price

price1 = Price(100.5, precision=1) # 1 decimal place
price2 = Price(0.125, precision=3) # 3 decimal places

result = price1 + price2
print(result)          # 100.625
print(result.precision) # 3 (max of 1 and 3)
```



Type-specific constraints

Quantity

`Quantity` represents non-negative amounts. Attempting to create a negative quantity or subtract a larger quantity from a smaller one raises an error:

```
from nautilus_trader.model.objects import Quantity

# This raises ValueError: Quantity cannot be negative
qty = Quantity(-100, precision=0)

# This also raises ValueError
qty1 = Quantity(50, precision=0)
qty2 = Quantity(100, precision=0)
result = qty1 - qty2 # Would be -50, which is invalid
```

Money

`Money` values include a currency. Addition and subtraction between `Money` values require matching currencies:

```
from nautilus_trader.model.objects import Money
from nautilus_trader.model.currencies import USD, EUR

usd_amount = Money(100.00, USD)
eur_amount = Money(50.00, EUR)

# This works - same currency
result = usd_amount + Money(25.00, USD)

# This raises ValueError - currency mismatch
result = usd_amount + eur_amount
```

Common patterns

Accumulating values

Since value types are immutable, accumulate by reassigning:

```
from nautilus_trader.model.objects import Money
from nautilus_trader.model.currencies import USD

total = Money(0.00, USD)
amounts = [Money(100.00, USD), Money(50.00, USD), Money(25.00, USD)]
```

```
for amount in amounts:
    total = total + amount # Reassign to new Money instance

print(total) # 175.00 USD
```

Converting to other types

Value types provide conversion methods:

```
from nautilus_trader.model.objects import Price

price = Price(123.456, precision=3)

# Convert to Decimal (preserves precision)
decimal_value = price.as_decimal()

# Convert to float
float_value = price.as_double()

# Convert to string
string_value = str(price) # "123.456"
```



Creating from strings

Parse value types from string representations:

```
from nautilus_trader.model.objects import Quantity, Price, Money

qty = Quantity.from_str("100.5")
price = Price.from_str("99.95")
money = Money.from_str("1000.00 USD")
```



< Synthetics
Previous Page

Data >
Next Page